

# Safe Semantic Planner — Design Report

## 1. Introduction

The Safe Semantic Planner is a C++ path-planning system designed to find a valid and low-cost route from an initial state to a goal state while avoiding designated unsafe states. The system represents the environment as states connected by transitions.

## 2. Objectives

The main objectives are safe path selection, avoidance of bad states, cost minimization, support for unavailable transitions, and reporting of planning performance metrics.

## 3. System Architecture

The system is divided into state representation, transition representation, planning-problem representation, planning-result representation, safety calculations, and the LPAStar planner. The command-line main program collects input and displays results.

## 4. Planning Workflow

Input states and transitions are stored in a PlanningProblem. The planner initializes path costs, evaluates available transitions, rejects transitions entering bad states, relaxes reachable states, records parent states and transitions, and reconstructs the final path.

## 5. Safety

Bad states are explicitly excluded from selected paths. The Safety component also calculates a minimum geometric safety distance using state coordinates.

## 6. Metrics

The result includes total path cost, minimum safety distance, an explored-state planning-work metric, planning time in milliseconds, bad states visited, and an estimate of memory used by the principal planner data structures.

## 7. Experimental Results

| Test                         | Path             | Cost  | Safety | Bad Visited | Time (ms) | Memory (KB) |
|------------------------------|------------------|-------|--------|-------------|-----------|-------------|
| Test 1 - Basic path          | 1 -> 2 -> 4      | 5.00  | inf    | 0           | 0.35      | 0.00        |
| Test 2 - Bad-state avoidance | 1 -> 4           | 10.00 | 1.41   | 0           | 0.03      | 0.19        |
| Test 3 - Safe path           | 1 -> 4 -> 5 -> 6 | 5.00  | 1.41   | 0           | 0.04      | 0.31        |

Tests 1–3 were re-run after the metrics were added and passed. Additional development tests covering transition availability, goal changes, and transition changes also passed during development.

## 8. Limitations and Future Work

The current planner uses an incremental relaxation approach inspired by LPA\* rather than the complete textbook LPA\* OPEN priority queue and key-update procedure. Future work can add the full LPA\* mechanism, graph visualization, a web frontend, larger benchmarks, and algorithm comparisons.

## 9. Conclusion

The project demonstrates safe, cost-aware path planning with explicit unsafe-state avoidance and measurable planning performance. The current implementation provides a functional command-line submission baseline that can be extended with a graphical or web frontend.